WEEK-1 Python REFRESHER

Day 1

Python is a high-level programming language known for its simple and readable syntax.
Features:
- Clean code writing with fewer lines
- Widely used in web development, automation, data analysis, ai and many other fields
- Dynamically typed and has automatic garbage collection (language automatically cleans up memory it is not using, so no need to free it up manually)
- Known for its readability, which means code is easier to write, understand and maintain.
- Large library support; ready-made modules for web, data science, ai, automation, etc.

How python programs are executed:

1. **Writing the source code** —> writing the code in a "***.py"*** file.
2. **Compilation to byte-code** —> Python interpreter first translates the source code into an intermediate form "byte-code". This is saved in "***.pyc***" file inside the "***pycache***" folder.
3. **Python Virtual Machine (PVM)** —> the byte-code is then given to the PVM which reads and executes the code instruction by instruction (actual interpreting step)
4. **Output** —> the PVM talks to the OS and hardware, and the result is shown to the user.

Source code (.py) —> Interpreter (compiles) —> Byte-code (.pyc) —> PVM/Interpreter (executes) —> Output

Source code:
The human-readable instructions we write (.py file). The computer cannot understand it directly
Interpreter:
The program that reads the source code and processes it. It first converts the source code into byte-code, then hands that byte-code to the virtual machine to run.
Byte-code:
A simple, compact set of instructions in between the source code and machine code. It is not for humans and is not tied to any specific processor. Python saves it in "***.pyc***" file.
Python Virtual Machine (PVM):
The software that reads the byte-code one instruction at a time and carries it out, using the real CPU and OS underneath. It is the part that actually runs the program.

Analogy:
- Source code: a book in English
- Interpreter: translator that converts it to a simple shorthand
- Byte-code: the shorthand notes
- PVM: the person who reads the shorthand notes and acts on it

Variables:

They are used to store data that can be referenced and manipulated during program execution.
- Python variables do not require explicit declaration of variable type.
- Type of variable is inferred based on the value assigned.

Eg: x = 10
Here, object '10' is created in memory, and the name 'x' is attached to it.

Object reference:

A reference is the link between a variable name and the object in memory. Python variables are references, not boxes that store values.

- No type declaration
- Variables can be reassigned to objects of different types

```
x = 10
x = "hello"   # x now points to a string object
```

```
a = [1, 2, 3]
b = a          # b refers to the same list
b.append(4)
print(a)       # [1, 2, 3, 4]
```

Dynamic typing:

In Python, there is no need to declare the type of a variable. The type is decided at runtime and belongs to the object, not the variable name.

```
x = 10        # x refers to an int
x = "hello"   # now x refers to a str
x = [1, 2]    # now x refers to a list
```

—> the same name can point to different objects of different types at different times.

Python is dynamically-typed, but is also ***strongly typed***. It does not mix incompatible types.

```
print("5" + 3)   # TypeError: can only concatenate str to str
```

Type checking:

It means verifying that an operation is used on the correct type. It happens at *runtime*, when the line is executed.
The “type()” function returns the type of object at runtime. It is used to identify the data type of variables and objects during program execution.

```
x = 10
print(type(x))              # <class 'int'>
print(isinstance(x, int))   # True
```

Built-in data types:

Data types define the type of values stored in variables and determine the operations that can be performed on that data.

1. **int**: value is represented by ‘***int***’ class. It contains positive/negative whole numbers.
2. **float**: value is represented by ‘***float***’ class. It is a real number with a floating point representation; specified with a decimal point.
3. **bool**: represents one of the 2 values - True/False. Is is used in conditions and comparisons. Represented by the ‘***bool***’ class.
4. **str**: they are used to store text data. represented using the ‘***str***’ class and can be created using single, double or triple quotes.

5. **NoneType**: it is type of the special value "***None***", which represents '*no value*' or '*nothing here*'.

Type conversion:

It is the process of changing a value from one data type to another.

- Helps ensure correct operations and calculations
- Python supports ***two types*** of conversions - ***Implicit*** and ***Explicit***

1. **Implicit type conversion**: it happens automatically when different data types are used together in an expression.

• Python converts smaller data types to bigger when needed
• Conversion happens at runtime to keep results accurate

```
x = 10          # Integer
y = 10.6        # Float
z = x + y

print("x:", type(x))
print("y:", type(y))
print("z =", z)
print("z :", type(z))
```

Output

```
x: <class 'int'>
y: <class 'float'>
z = 20.6
z : <class 'float'>
```

- "x" is an integer, "y" is a float.
- During 'z=x+y', python implicitly converts "x" to float to avoid data loss.
- As a result, "z" becomes a float number.

2. **Explicit type conversion**: also called as type casting, is when a programmer manually changes a value from one data type to another.

• Done using build-in functions like **int()**, **float()**, **str(), bool()**
• Gives full control over how data is interpreted or processed
• Used when automatic conversion is not suitable

```
s = "100"  # String
a = int(s)
print(a)
print(type(a))
```

Output

```
100
<class 'int'>
```

- “s” is a string with value “100”.
- Using int(s), string is explicitly converted into an integer.

Common type conversion errors:

- The value doesn’t match the target type. Eg: int(“abc”)
- Converting a decimal string directly to int. Eg: int(“3.5”)
- Mixing incompatible types in an operation. Eg: str + int)
- Data loss; converting float to int cuts the decimal part instead of rounding.

Operators:

Operators are used to perform operations on values and variables.

- Operators: special symbols like +, -, *, /, etc.
- Operands: values on which operator is applied.

1. **Arithmetic operators**: they are used to perform basic mathematical operations.

```python
a = 15
b = 4

print("Addition:", a + b)

print("Subtraction:", a - b)

print("Multiplication:", a * b)

print("Division:", a / b)

print("Floor Division:", a // b)

print("Modulus:", a % b)

print("Exponentiation:", a ** b)
```

Output

```
Addition: 19
Subtraction: 11
Multiplication: 60
Division: 3.75
Floor Division: 3
Modulus: 3
Exponentiation: 50625
```

2. **Comparison operators**: they compare values. either returns True or False according to the condition.

```
a = 13
b = 33

print(a > b)
print(a < b)
print(a == b)
print(a != b)
print(a >= b)
print(a <= b)
```

```
Output

False
True
False
True
False
True
```

3. **Logical operators**: perform logical AND, OR, NOT operations. Used to combine conditional statements.

- AND: True only if both conditions are True
- OR: True if at least one condition is True
- NOT: reverses the result; True <-> False

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
```

```
Output

False
True
False
```

4. **Assignment operators**: they are used to assign values to variables. It is used to assign the value of the right side of the expression to the left side.

```
a = 10
b = a
print(b)
b += a
print(b)
b -= a
print(b)
b *= a
print(b)
b <<= a
print(b)
```

Output

```
10
20
10
100
102400
```

5. **<u>Membership operators</u>**: they are used to test if a value/variable is in a sequence. "***in***" and "***not in***" are the operators.

```
x = 24
y = 20
my_list = [10, 20, 30, 40, 50]

if (x not in my_list):
    print("x is NOT present in given list")
else:
    print("x is present in given list")

if (y in my_list):
    print("y is present in given list")
else:
    print("y is NOT present in given list")
```

Output

```
x is NOT present in given list
y is present in given list
```

6. **Identity operators**: they are used to check if two values are located on the same part of the memory. "***is***" and "***is not***" are the operators.

```
a = 10
b = 20
c = a

print(a is not b)
print(a is c)
```

Boolean evaluation:

Truthy values: these values evaluate to True in a Boolean context.

- Non-empty sequences/collections.
- Numeric values != 0
- Constant: True.

```
if [1, 2]:
    print("Non-empty list is truthy")

if -4:
    print("-4 is truthy")
```

Output

```
Non-empty list is truthy
-4 is truthy
```

Falsy values: these values evaluate to False in a Boolean context.

- Empty sequences/collections
- Numbers including 0, float and complex
- Constants: None, False

```
if not 0:
    print("0 is falsy")

if not []:
    print("Empty list is falsy")
```

Output

```
0 is falsy
Empty list is falsy
```

Built-in bool() function:

One can check if a value is either truth/falsy with the '***bool()'*** function. This function is used return/ convert a value to a boolean value.

- Truthy values are converted to True, Falsy values are converted to False.

```
print(bool(7))
print(bool(0))
print(bool([1,2,3]))
print(bool([]))
print(bool(None))
```

Output

```
True
False
True
False
False
```

Mutable v/s Immutable objects:

Mutable	Immutable
Can be changed after creation	Can't be changed after creation
Memory address on change stays the same	A new object is created
Change through one name shows in all names	Other names are not affected
Suitable for data that changes frequently	Suitable for fixed/constant data
Eg: list, dict, set	Eg: tuple, str, float, int, bool

Python object equality:

'==' and 'is' are comparison operators that serve different purposes.

- '**==**' checks whether two objects have equal values
- '**is**' checks whether two variables reference the same object in memory

Two objects can contain identical values without sharing the same memory location.

```
a = [1,2,3]
b = [1,2,3]

print(a == b)
print(a is b)
```

Output

```
True
False
```

—> ‘a == b’ checks value equality and returns True.
—> ‘is’ checks memory identity and returnsFalse.

==	is
Compares values	Compares object identity
Return ‘True’ when values are equal	Returns ’True’ when both variables reference the same object
Common use: comparing data	Common use: comparing with None or shared reference